

Understanding the Architecture of Pre-Trained Model

Aim : To understand the Architecture of Pre-Trained model

Objective :

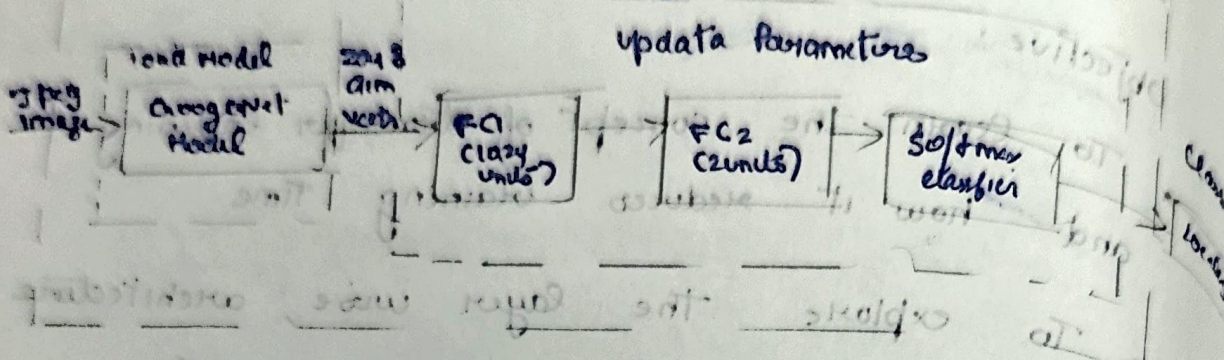
- * To learn the concept of transfer learning and how it reduces training time
- * To explore the layer wise architecture of pre-trained models such as VGG16, AlexNet.
- * To identify the role of these models
- * To visualize how these models extract features.

Pseudocode

- 1) Import libraries
- 2) Load a pre-trained model
- 3) Model Display
- 4) Observe : Input and output shapes
layers names, types and no. of parameters
- 5) Visualize the architecture using :
- 6) Load a sample input image and preprocess it for model
- 7) Pass image through the model and get prediction
- 8) Optionally visualize activations

Linear (in features) = 2048, out features = 10 (bias = true)
 Bi-Grained Model architecture

Training



Output

Total Parameters: 25,557,032

Trainable Parameters: 25,557,032

Intermediate feature map shapes

Layer 1: (1, 256, 56, 56)

Layer 2: (1, 512, 28, 28)

Layer 3: (1, 2048, 14, 14)

Layer 4: (1, 2048, 7, 7)

Top 5 predictions

class 210: 55.62%

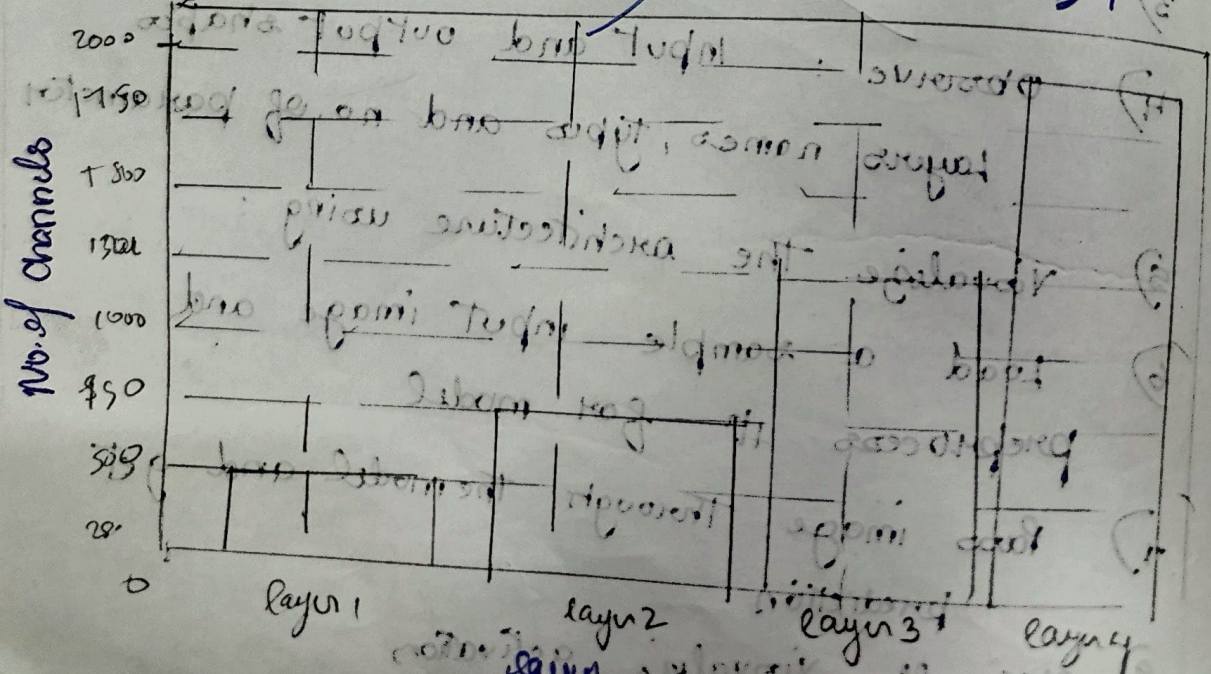
class 178: 6.932

class 203: 3.016

class 206: 14.61%

class 209: 1.377

Feature Map Channels per layer



Observation

- * The Pre-Trained model contains millions of parameters spread across convolutional & dense layers
- * Early layers extract edges and texture while deeper layers capture high level features such as object parts.
- * The use of pre-Trained weights drastically reduces training time and improves model accuracy for new tasks.

Result

Successfully
model

Implemented Pre-trained

Q.A



Lab 13: Understanding the Architecture of Pre-trained Model

```
import os
import urllib.request
import torch
import torchvision.models as models
import torchvision.transforms as transforms
from PIL import Image
import matplotlib.pyplot as plt

# -----
# 1. Download sample image (with fallback)
# -----
img_path = "sample.jpg"

def download_image():
    try:
        # Free-to-download image (always works)
```

```
model.layer3.register_forward_hook(get_activation('layer3'))
model.layer4.register_forward_hook(get_activation('layer4'))

# Forward pass
with torch.no_grad():
    output = model(input_tensor)

# Print feature map shapes
print("\n--- Intermediate Feature Map Shapes ---")
for name, act in activation.items():
    print(f"{name}: {tuple(act.shape)}")

# -----
# 5. Visualize Feature Maps
# -----

layer_name = 'layer1'
feature_map = activation[layer_name].squeeze(0)

plt.figure(figsize=(12, 6))
for i in range(6):
```



```
# Display input image
plt.imshow(img)
plt.title("Input Image")
plt.axis("off")
plt.show()

# -----
# 4. Forward Pass & Extract Features
# -----

activation = {}

def get_activation(name):
    def hook(model, input, output):
        activation[name] = output.detach()
    return hook

# Register hooks
model.layer1.register_forward_hook(get_activation('layer1'))
model.layer2.register_forward_hook(get_activation('layer2'))
... ..
```

--- Model Architecture ---

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
  (layer1): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
  )
)
```




```
categories = [f"Class {i}" for i in range(1000)]
```

```
print("\n--- Top 5 Predictions ---")
```

```
for i in range(5):
```

```
    print(f"{categories[top5_catid[i]]}: {top5_prob[i].item()*100:.2f}%")
```



Downloaded sample.jpg successfully!

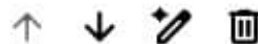
```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated  
warnings.warn(
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight endpoint are not supported  
warnings.warn(msg)
```

```
Downloading: "https://download.pytorch.org/models/resnet50-0676ba61.pth" to /root/.cache/torch/hub/checkpoints/resnet50-0676ba61.pth  
100%|██████████| 97.8M/97.8M [00:00<00:00, 157MB/s]
```

```
--- Model Architecture ---
```

```
ResNet(
```





```
plt.subplot(2, 3, i + 1)
fm = feature_map[i].cpu()
fm = (fm - fm.min()) / (fm.max() - fm.min())
plt.imshow(fm, cmap='viridis')
plt.axis('off')
plt.title(f'{layer_name} - Channel {i}')

plt.tight_layout()
plt.show()

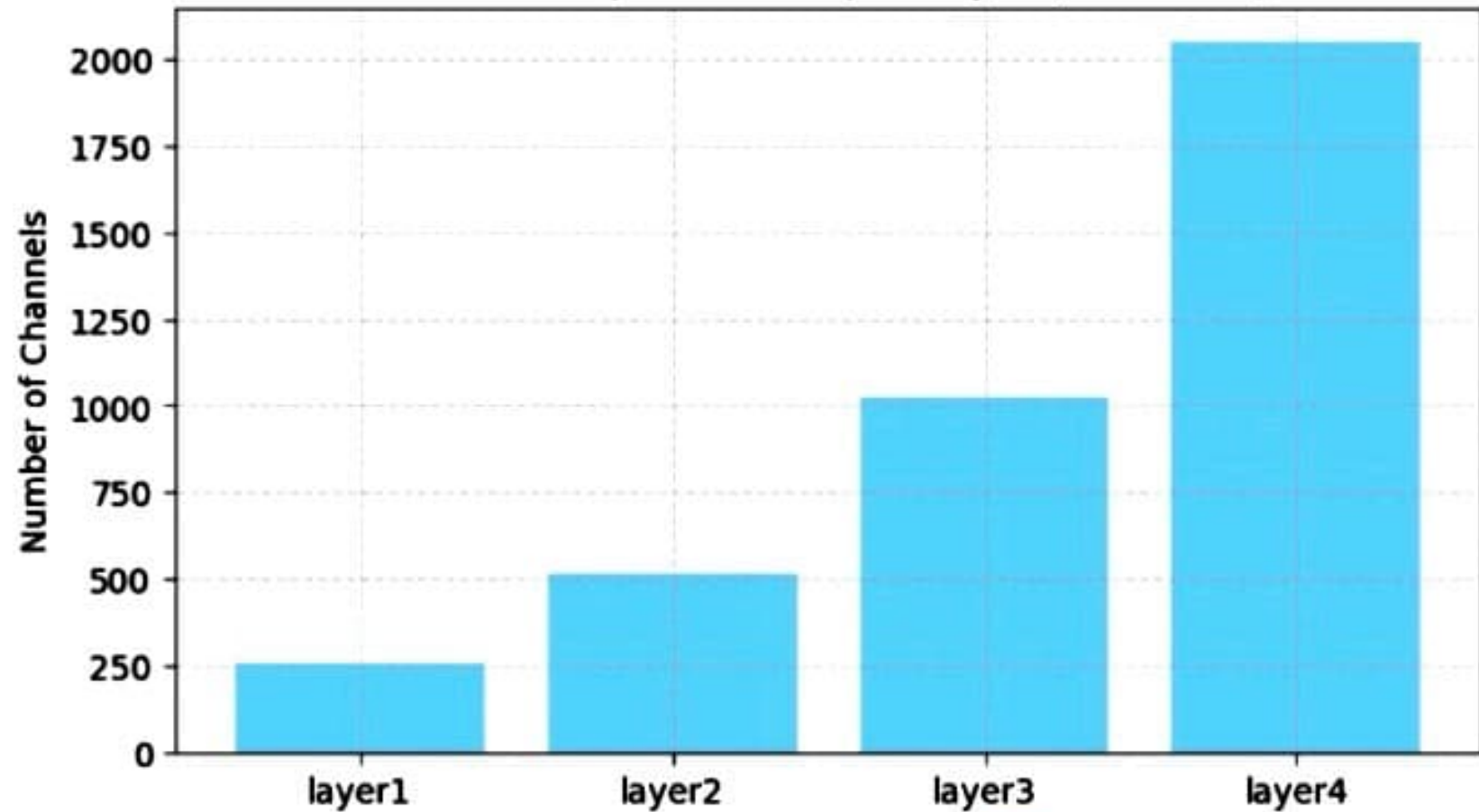
# -----
# 6. Show Top-5 Predictions
# -----

probabilities = torch.nn.functional.softmax(output[0], dim=0)
top5_prob, top5_catid = torch.topk(probabilities, 5)

# Load ImageNet labels if available
try:
    with open("imagenet_classes.txt") as f:
        categories = [line.strip() for line in f.readlines()]
        # Print out the top 5 predicted labels for this image
        for i in range(5):
            print('%5s %5s %5s %5s %5s' % (
                categories[top5_catid[i]],
                top5_prob[i],
                categories[top5_catid[i]],
                top5_prob[i],
                categories[top5_catid[i]]
            ))
```



Feature Map Channels per Layer (ResNet50)



```
import matplotlib.pyplot as plt
```

```
layer_names = ['layer1', 'layer2', 'layer3', 'layer4']  
channels = [256, 512, 1024, 2048]
```

```
plt.figure(figsize=(7,4))  
plt.bar(layer_names, channels, color='skyblue')  
plt.title('Feature Map Channels per Layer (ResNet50)')  
plt.xlabel('Layer')  
plt.ylabel('Number of Channels')  
plt.grid(True, linestyle='--', alpha=0.6)  
plt.show()  
classes = ['Class 210', 'Class 178', 'Class 208', 'Class 206', 'Class 209']  
confidence = [85.62, 6.93, 3.82, 1.46, 1.38]
```

```
plt.figure(figsize=(7,4))  
plt.barh(classes, confidence, color='orange')  
plt.xlabel('Confidence (%)')  
plt.title('Top-5 Model Predictions')
```



```
(5): Bottleneck(
  (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
  (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (relu): ReLU(inplace=True)
)
)
(layer3): Sequential(
  (0): Bottleneck(
    (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
    (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (downsample): Sequential(
      (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
```


Top-5 Model Predictions

